PROGRAMMATION

ERREURS À ÉVITER ET BONNES PRATIQUES

No notes on this slide.

PROGRAMMATION

L'objectif

QUEL EST L'OBJECTIF D'ÉCRIRE UN CODE PROPRE ?

No notes on this slide.

PROGRAMMATION

L'objectif

Cela permet

- aux autres d'utiliser notre code
- à sois-même, lorsqu'on aura besoin de le modifier
- d'éviter des erreurs de logique et de bugs
- d'être plus efficace dans le futur, pour ajouter des fonctions ou corriger des bugs

On produit du code de qualité pour les humains, pas pour la machine

Ces raisons sont suffisantes pour toujours produire du code respectant les standards de qualité. Un code difficile à comprendre sera souvent ignoré, lui faisant perdre beaucoup de son intérêt.

PROGRAMMATION

La qualité

« Rendez les choses aussi simples que possible, mais pas plus simples. »

Albert Einstein

Plus concrètement :

Tout code doit être spontanément évidemment juste

Au delà des belles phrase, l'objectif final d'un code de qualité est d'être facilement compréhensible par les humains. En particulier, on ne cherchera pas à optimiser ou anticiper plus que nécessaire. Un bon code doit être aussi simple et direct que possible, l'intention doit être claire et il doit être évident que le code accomplit ce qu'il est censé faire sans erreur. Il se trouve que cela est plus difficile à faire qu'il n'y parait... On connaît généralement ce problème sous l'acronyme K.I.S.S : Keep It Simple, Stupid

PROGRAMMATION

LES PROBLÈMES RÉCURENTSS

No notes on this slide.

PROGRAMMATION

Les problèmes récurentss

Problème n° 1 : Dépendances et code spaghetti

```
def get_x_y_centered(xs, ys, side, min_y_extent):
  if len(xs) <= 0:
    return np.empty(0)
  x_min, y_min = np.min(xs), np.min(ys)
  x_extent = np.max(xs) - x_min
  y_extent = max(np.max(ys) - y_min, min_y_extent)
  resolution = x_extent / (side - 1) if x_extent > y_extent else y_extent / (side - 1)
  if resolution <= 0:
    return np.empty(0)
  xs_centered = ((xs - x_min + (side * resolution - x_extent) / 2.0) / resolution)
  ys_centered = ((ys - y_min + (side * resolution - y_extent) / 2.0) / (-resolution))
  return np.column_stack((xs_centered, ys_centered)).astype(np.int32)
```


No notes on this slide.

PROGRAMMATION

Les problèmes récurentss

Problème n° 1 : Dépendances et code spaghetti

```
def get_x_y_centered(xs, ys, side, min_y_extent):
    if len(xs) <= 0:
        return np.empty(0)
    x_min, y_min = np.min(xs), np.min(ys)
    x_extent = np.max(xs) - x_min
    y_extent = max(np.max(ys) - y_min, min_y_extent)
    resolution = x_extent / (side - 1) if x_extent > y_extent else y_extent / (side - 1)
    if resolution <= 0:
        return np.empty(0)
    xs_centered = ((xs - x_min + side * resolution - x_extent) / 2.0) / resolution
    ys_centered = ((ys - y_min + side * resolution - y_extent) / 2.0) / (-resolution)
    return np.column_stack((xs_centered, ys_centered)).astype(np.int32)
```

No notes on this slide.

PROGRAMMATION

Les problèmes récurrents

Problème n° 1 : Dépendances et code spaghetti

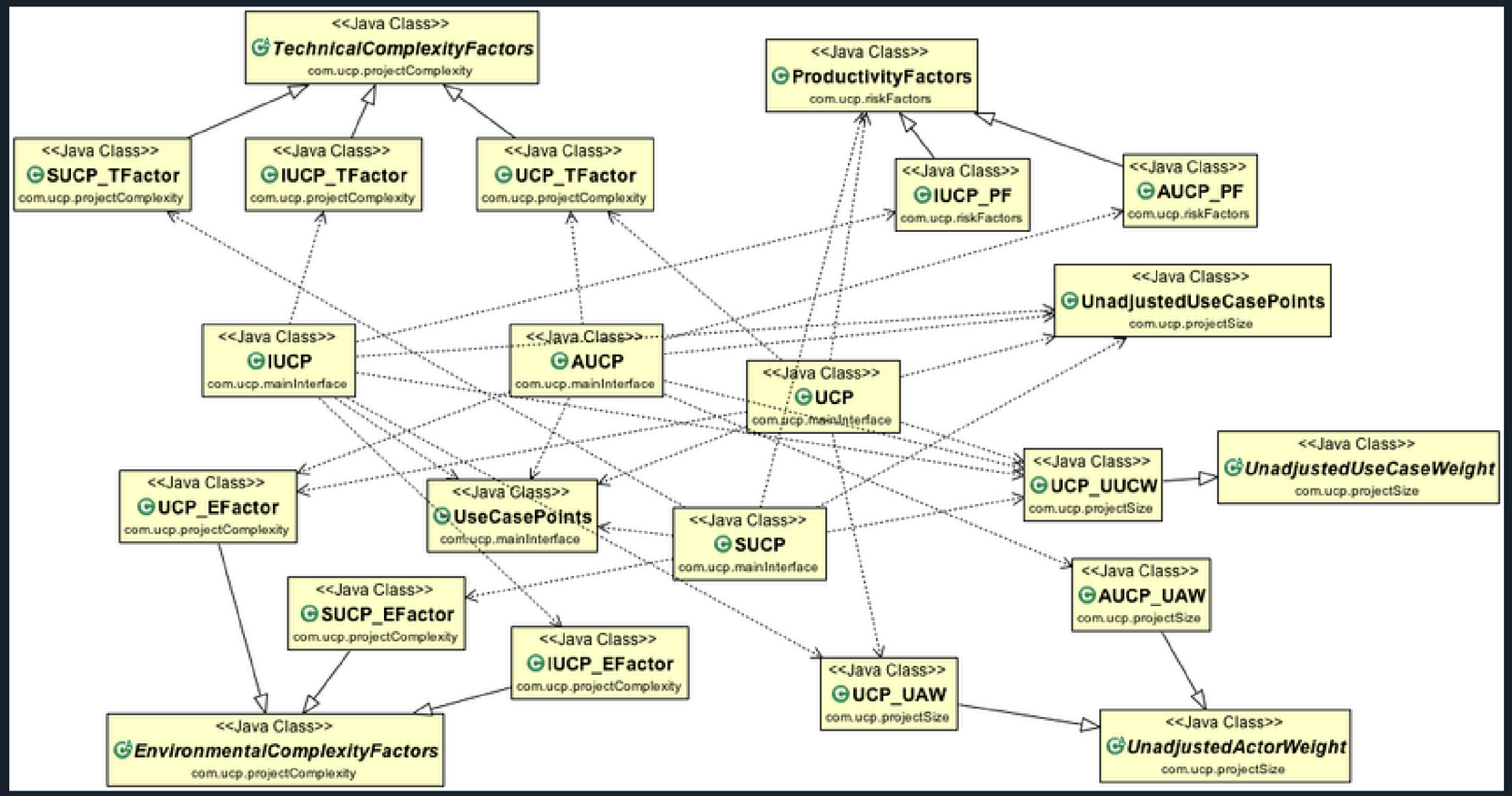
```
def get_x_y_centered(xs, ys, side, min_y_extent):
    if len(xs) <= 0:
        return np.empty(0)
    x_min, y_min = np.min(xs), np.min(ys)
    x_extent = np.max(xs) - x_min
    y_extent = max(np.max(ys) - y_min, min_y_extent)
    resolution = x_extent / (side - 1) if x_extent > y_extent else y_extent / (side - 1)
    if resolution <= 0:
        return np.empty(0)
    xs_centered = ((xs - x_min + (side * resolution - x_extent) / 2.0) / resolution)
    ys_centered = ((ys - y_min + (side * resolution - y_extent) / 2.0) / (-resolution))
    return np.column_stack((xs_centered, ys_centered)).astype(np.int32)
```

No notes on this slide.

PROGRAMMATION

Les problèmes récurrents

Problème n° 1 : Dépendances et code spaghetti



https://www.researchgate.net/figure/Class-Dependencies-of-Like-UCP-Program_fig2_329420327

No notes on this slide.

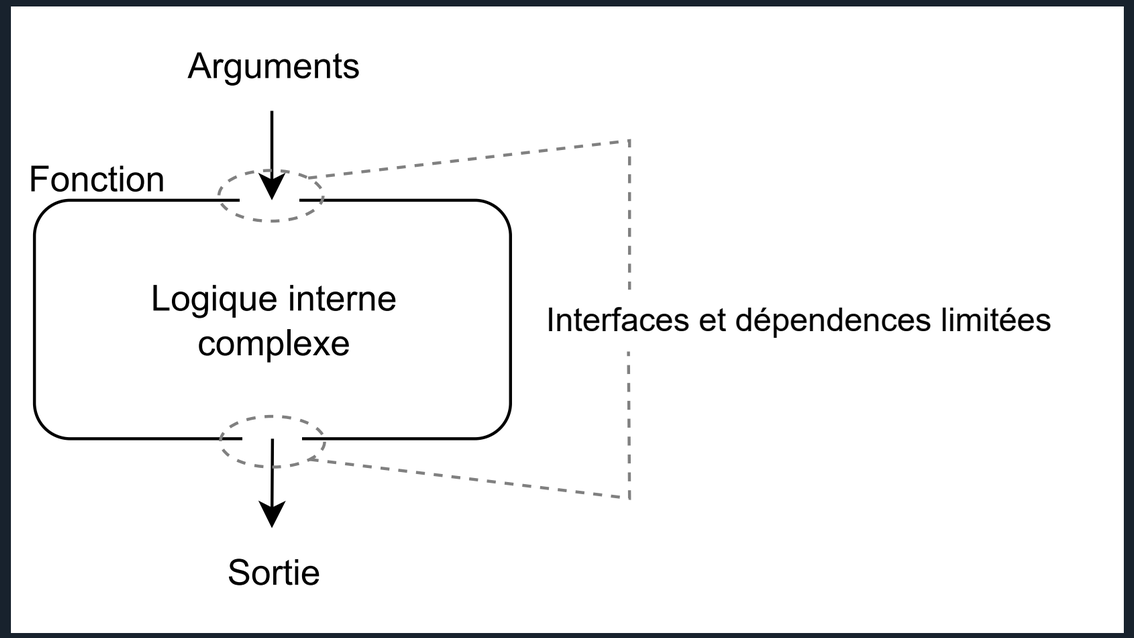
PROGRAMMATION

Les problèmes récurrents

Problème n° 1 : Dépendances et code spaghetti

Les solutions :

- Isoler le code / la logique en plusieurs parties → Fonctionnaliser



Une des meilleures manière d'isoler du code est de le mettre dans une fonction dédiée.
Mais il faut faire attention à la manière de fonctionnaliser. On voudra des fonctions avec un minimum d'arguments et un minimum de valeurs de retour, afin de réduire les dépendances avant et après la fonction.

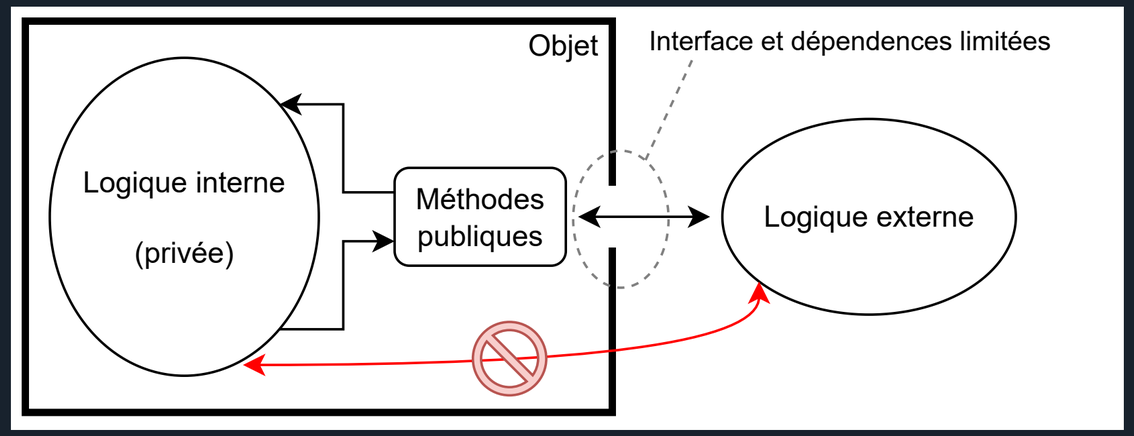
PROGRAMMATION

Les problèmes récurrents

Problème n° 1 : Dépendances et code spaghetti

Les solutions :

- Isoler le code / la logique en plusieurs parties → Fonctionnaliser
- L'encapsulation



L'encapsulation en programmation orientée objet vise le même principe : en exposant uniquement un sous-ensemble de fonction de l'objet pour communiquer avec l'extérieur, on s'assure peu de dépendances avec l'extérieur. L'encapsulation aussi d'autres avantages. Si on souhaite modifier la logique interne de l'objet, on se garanti de pouvoir le faire sans avoir à adapter d'autres partie du code.

PROGRAMMATION

Les problèmes récurrents

Problème n° 1 : Dépendances et code spaghetti

Les solutions :

- Isoler le code / la logique en plusieurs parties → Fonctionnaliser
- L'encapsulation
- Séparer son code en plusieurs fichiers

Séparer son code est aussi une bonne pratique qui incite à ne pas créer trop de dépendances entre fichiers. Cela facilite aussi la lecture du code, on se perd rapidement dans un fichier trop long.

PROGRAMMATION

Les problèmes récurrents

Voir :

- Principe K.I.S.S.
- Principe de la responsabilité unique
- Principe de la séparation des préoccupations
- L'encapsulation
- Principes S.O.L.I.D.
- Loi de Déméter

No notes on this slide.